MODULE II • LECTURE 03

Dropout & Network Design Principles

Dr Tamal Ghosh

Capacity • Overfitting • Regularization • Architecture

Recap: From Neuron to Network

THE NEURON

A Single Neuron Computes:

$$z = Wx + b$$

Followed by a non-linear activation:

$$a = f(z)$$

THE NETWORK

A Feedforward Network

Stacks these operations sequentially to form a deep architecture:

$$x \rightarrow \text{Linear} \rightarrow \text{Activation} \rightarrow \text{Linear} \rightarrow \text{Output}$$

Today we focus on how to design this network properly.

Learning Objectives

By the end of this lecture, students should be able to:

- Explain model **capacity**.
- Distinguish **underfitting** and **overfitting**.
- Explain why neural networks overfit.
- Define **regularization**.
- Explain **dropout**.
- Understand training vs inference behavior with dropout.
- Identify important network-design decisions.
- Relate architecture choices to NLP classification.

What Is Model Capacity?

Model capacity refers to the ability of a model to **represent complex functions**.

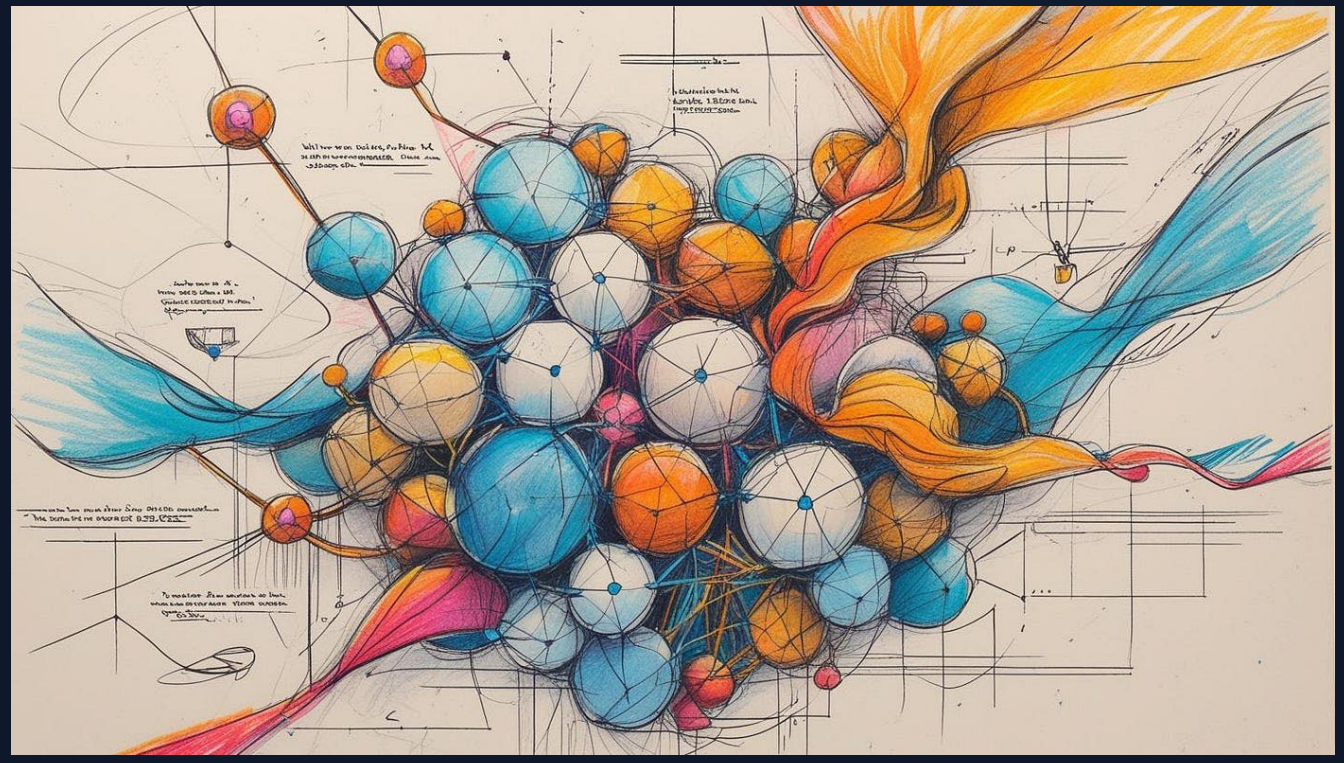
A simple model has limited capacity:

$$y = wx + b$$

A network generally has **greater capacity** with:

- More layers
- More neurons
- More parameters

⚠**But more capacity is not automatically better.**



Capacity: Too Little vs Too Much

Too Little

↓

Cannot learn the pattern

↓

Underfitting

Appropriate

↓

Learns useful patterns

↓

Good Generalization

Too Much

↓

Can memorize training data

↓

Overfitting

The objective is not simply to maximize model complexity.

Underfitting

A model underfits when it is too simple to capture the underlying patterns.

Typical Symptoms

- High training error
- High validation error
- Poor performance even on training data

Example: Using a very small neural network to classify highly varied language data.

Main Causes

- Insufficient capacity
- Excessive regularization
- Poor features / data representation
- Insufficient training time

Overfitting

A model overfits when it learns the training examples too specifically.

Dataset	Performance
Training	Very high (almost perfect)
Validation	Much lower
Test	Much lower

The model has learned:

"What did I see?"

Rather than:

"What general pattern explains what I saw?"

NLP Example of Overfitting

TRAINING PHASE

"This movie was absolutely fantastic."

"An amazing and wonderful film."

Model learns strong associations:

- fantastic → positive
- amazing → positive

TESTING PHASE

"The movie was fantastic despite its terrible ending."

A model relying too heavily on individual words struggles here.

> **The Goal:** Learn generalizable patterns, not memorize examples.

Training Error vs Validation Error

During training, we track:

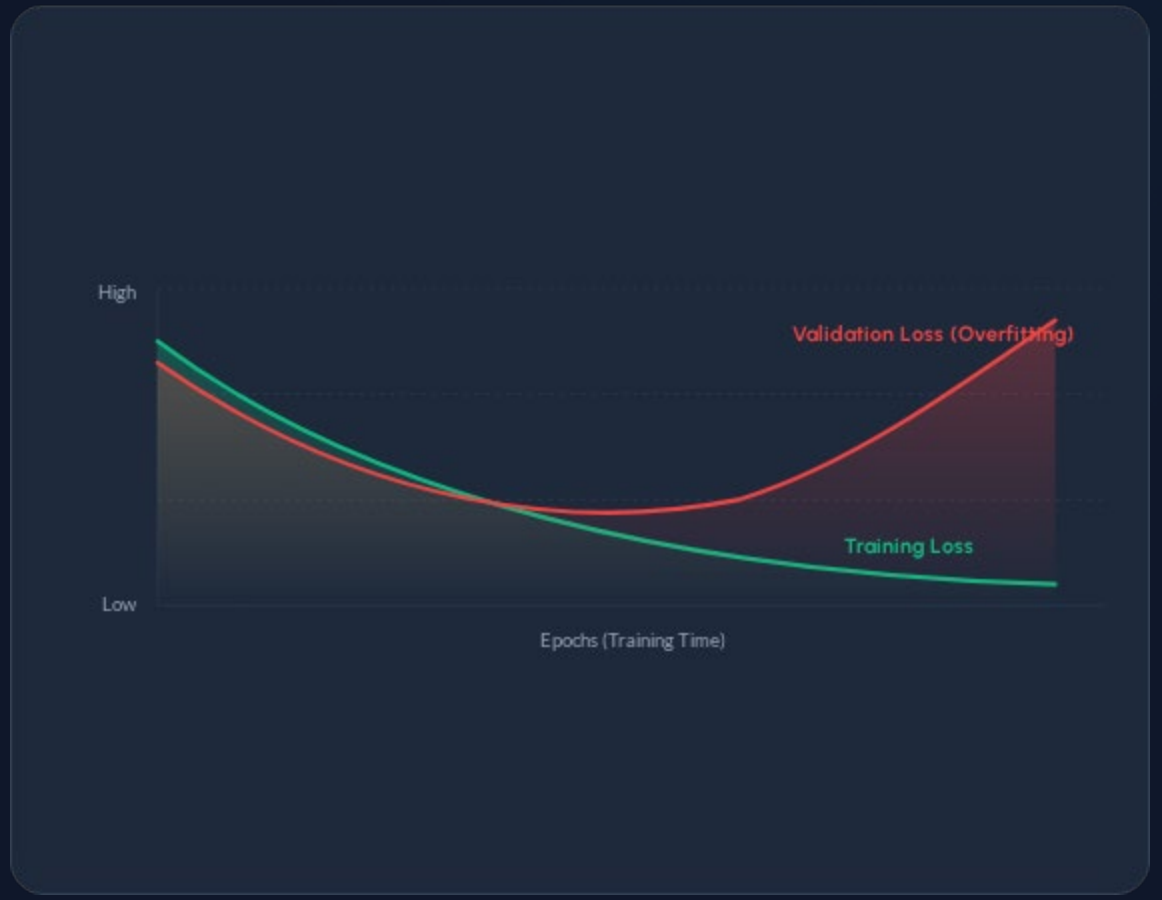
$$L_{train}$$

and validation:

$$L_{validation}$$

- Training loss keeps decreasing
- Validation loss decreases, hits a minimum, then **starts increasing**.

Divergence is a classic warning sign of overfitting.



Generalization

A model should perform well on unseen data.

The real objective is not alone:

$$\min L_{train}$$

We want a model that performs well on the **underlying data distribution**.

Key Idea: Training performance is necessary but not sufficient.

What Is Regularization?

Regularization introduces a mechanism that discourages undesirable model complexity.

Standard Optimization

Instead of minimizing only:

$$L(\theta)$$

(Focuses purely on memorizing training data)

Regularized Optimization

We minimize:

$$L(\theta) + \lambda R(\theta)$$

L(θ) = Training loss

R(θ) = Regularization term

λ = Regularization strength

Why Regularize?

✕ Without Regularization

Training data

↓

Very flexible model

↓

Can memorize examples

↓

Poor Generalization

✓ With Regularization

Training data

↓

Constrained model

↓

Learns robust patterns

↓

Better Generalization

Common Regularization Strategies

Neural networks can be regularized through multiple approaches:

- L1 Regularization
- L2 Regularization / Weight Decay
- **Dropout**
- Early Stopping
- Data Augmentation
- Reducing Network Size
- Increasing Training Data

Today we focus particularly on: **Dropout**

What Is Dropout?

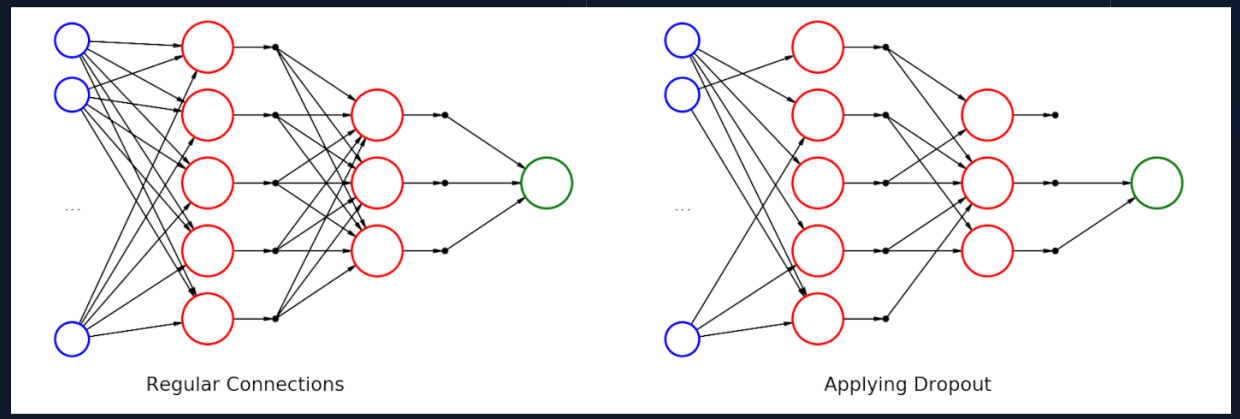
Dropout is a regularization technique in which some neuron activations are **randomly dropped** during training.

CONCEPTUALLY

Before dropout:

After randomly dropping units:

Where ∘ represents an inactive unit.



Why Dropout?

The Problem

Imagine a network always relying heavily on one particular neuron.

The network may develop a **dependency** on specific features or pathways (co-adaptation).

The Solution

Dropout prevents the network from becoming too dependent on particular neurons.

Result:
The model is encouraged to develop more distributed representations.

Dropout Probability

Defining 'p'

Let p = dropout probability.

For example, **p = 0.5** means approximately 50% of eligible activations are randomly dropped during a training step.

Retention Probability

The complementary probability is:

$$q = 1 - p$$

where q is the probability of retaining an activation.

Dropout Example

Suppose a hidden layer contains the following activations:

```
[ 0.8, 0.4, 0.7, 0.2, 0.9 ]
```

Random Training Mask

With dropout, one possible mask generated could be:

```
[ 1, 0, 1, 0, 1 ]
```

Resulting Activations

The output sent to the next layer becomes approximately:

```
[ 0.8, 0, 0.7, 0, 0.9 ]
```

Important: The mask is randomly generated during training.

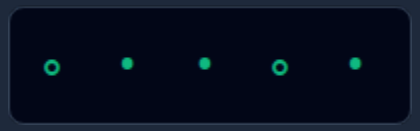
Dropout Is Random

Consider the same network across different training steps.

Step 1

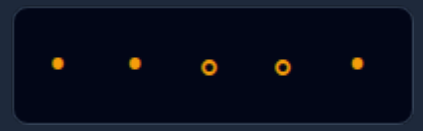
Subnetwork A is trained.

Step 2



Subnetwork B is trained.

Step 3



Subnetwork C is trained.

Different subnetworks are effectively trained across different steps. This creates a powerful regularizing effect (like an ensemble).

Dropout During Training

Standard Dropout

During training:

$$a' = m \odot a$$

a = activation vector

m = random dropout mask

$\odot$ = element-wise multiplication

Inverted Dropout

Activations are also scaled by the retention probability:

$$a' = \frac{m \odot a}{q}$$

This keeps the expected activation approximately unchanged, making inference easier.

Dropout During Inference

This distinction is crucial for deploying models.

TRAINING PHASE

Random neurons dropped

↓

Dropout Active

INFERENCE / TESTING

No random dropping

↓

Full Network Used

Therefore: Dropout is normally active during training and disabled during inference.

Dropout Intuition

Dropout encourages the network to avoid relying on a single pathway.

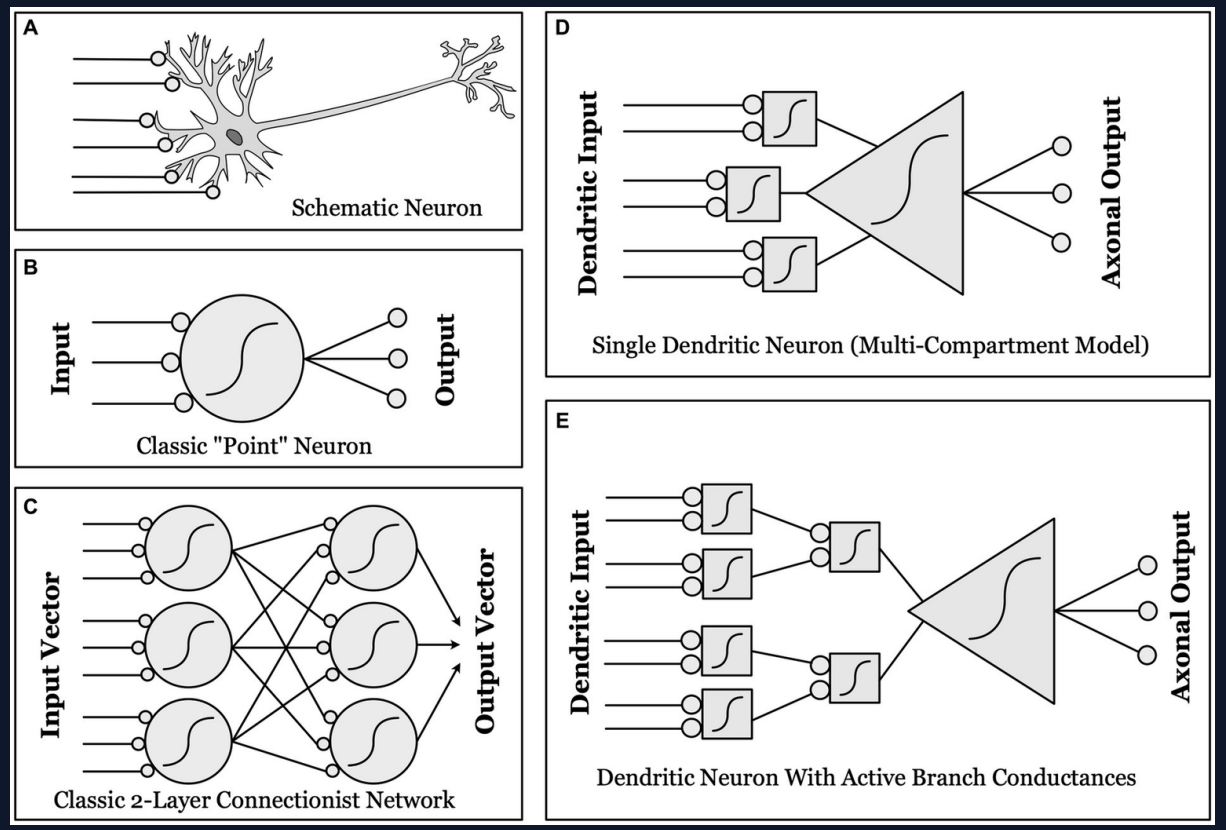
Instead of fragile links:

Feature A → Neuron X → Prediction

The network learns to use multiple signals:

```
Feature A ┐
Feature B ├→ Multiple Pathways → Prediction
Feature C ┤
Feature D ┘
```

This significantly improves robustness.



Network Design

What Is Network Design?

- Number of layers
- Number of neurons per layer
- Activation functions
- Output structure
- Dropout rate
- Learning rate
- Batch size
- Other architectural choices

There is no universally optimal architecture.

How Many Hidden Layers?

SHALLOW NETWORK

Input

↓

Hidden

↓

Output

DEEPER NETWORK

Input

↓

Hidden 1

↓

Hidden 2

↓

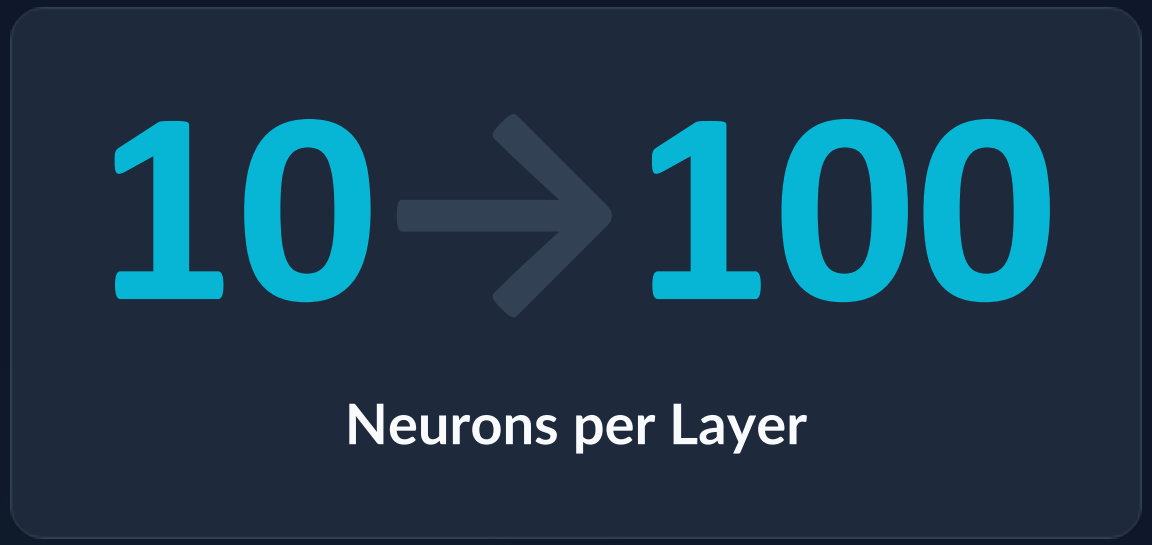
Hidden 3

↓

Output

More layers allow more transformations. But **deeper does not automatically mean better.**

How Many Neurons?



Scaling Up Width

Increasing neurons from 10 to 100 provides substantially greater representational capacity.

But it also means:

- More parameters
- More computation
- Potentially greater overfitting

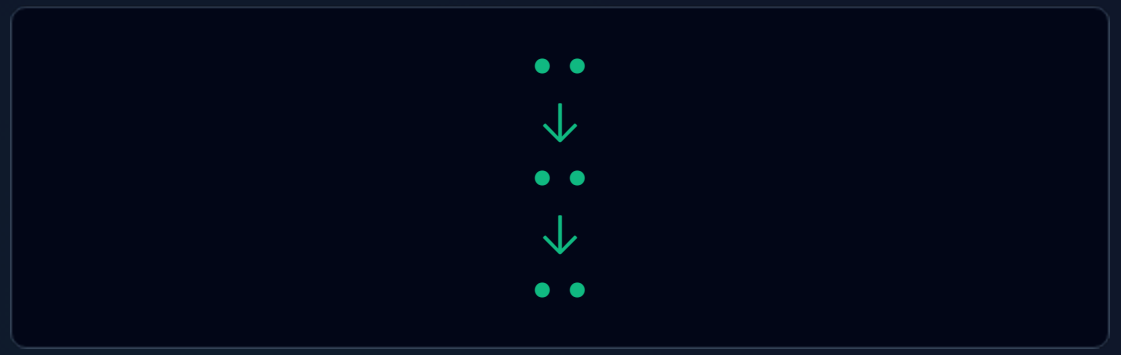
Use enough capacity to learn the task, but avoid unnecessary complexity.

Width vs Depth

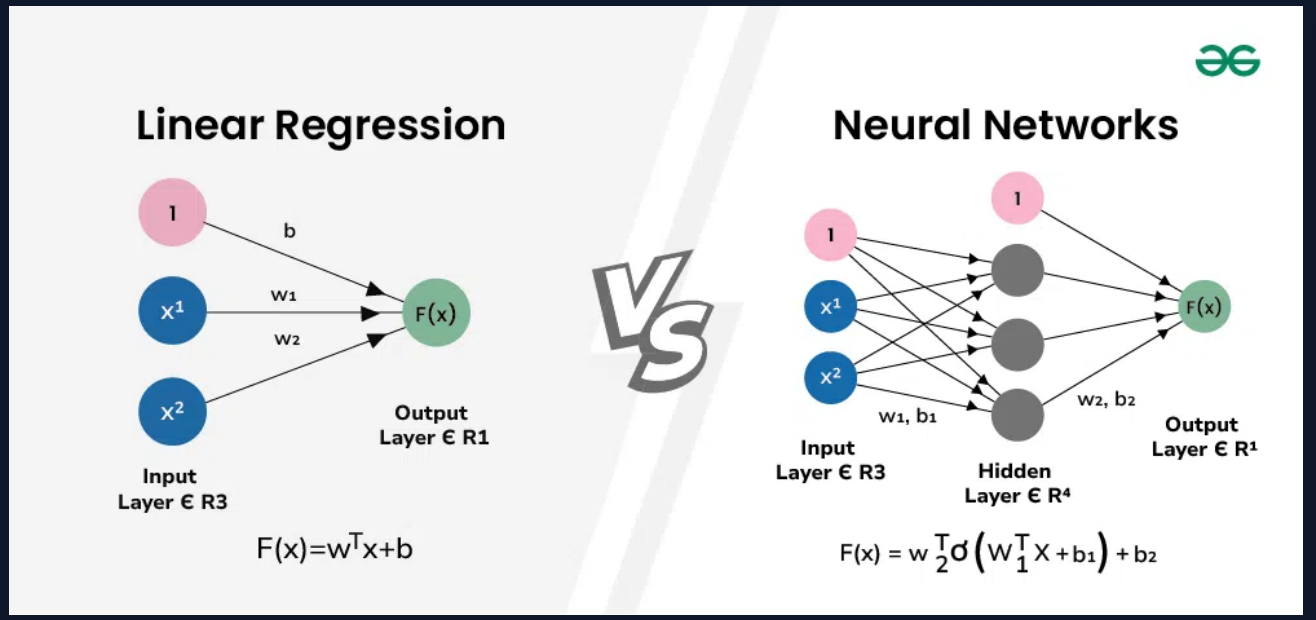
- **Width** = number of neurons per layer.
- **Depth** = number of layers.

Wide Network

Deep Network



Both increase capacity, but in different ways.



Parameter Count

Calculating the trainable capacity.

Architecture Specs

- **Input:** 100 features
- **Hidden layer:** 20 neurons
- **Output:** 1 neuron

The Math

Input → hidden (Weights + Bias):

$$100 \times 20 + 20 = 2020$$

Hidden → output (Weights + Bias):

$$20 \times 1 + 1 = 21$$

Total: **2041** parameters

Choosing Dropout

Dropout rate is a hyperparameter to be tuned.

$p = 0.1$

Light regularization. Good for small networks or low overfitting risk.

$p = 0.3$

Moderate regularization. A common starting point for many architectures.

$p = 0.5$

Aggressive regularization. Good for deep networks prone to heavy memorization.

Excessive dropout can make learning unnecessarily difficult.

Treat it as a design choice, not a universal rule.

Example: Sentiment Classification Architecture

Interpretation

- **Dense layers** learn transformations.
- **ReLU** introduces nonlinearity.
- **Dropout** regularizes the learned features.
- **Sigmoid** produces the binary output (0 to 1) for classification.

Practical Design Workflow

1. Start Simple

Build a basic architecture and train.

2. Evaluate

Check training + validation performance.

3. Adjust

Underfitting? Increase capacity. Overfitting? Add dropout.

4. Finalize

Tune architecture and test on held-out data.

Important: Do not blindly make the network deeper and wider.

Summary & Next Lecture

Key Takeaways

- **Capacity:** Layers/neurons increase ability to represent patterns.
- **Over/Under-fitting:** Balance between failing to learn vs memorizing.
- **Dropout:** Randomly removes activations to build robust pathways.

The Core Balance

Network design is about finding the optimal point:

$$\text{Capacity} \leftrightarrow \text{Generalization}$$

Next Lecture

Backpropagation & Optimization Techniques

We know what the network is and how to control its capacity. Next: How does the network actually learn its weights?